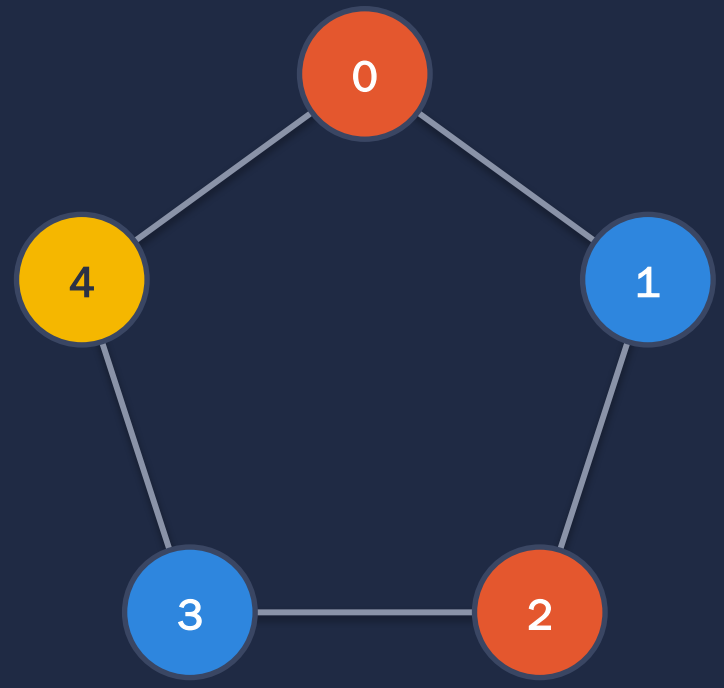
回溯搜索与图的三染色判定

一次 NP 完全问题的亲身体验

《计算机科学中的伟大思想》理论项目 · Basic 课堂展示

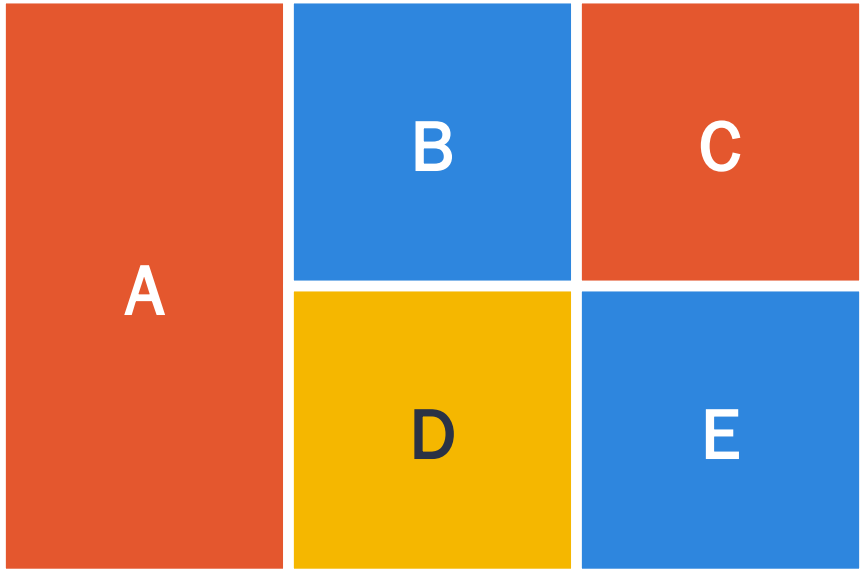
二染色 · 朴素回溯三染色 · 指数爆炸实验



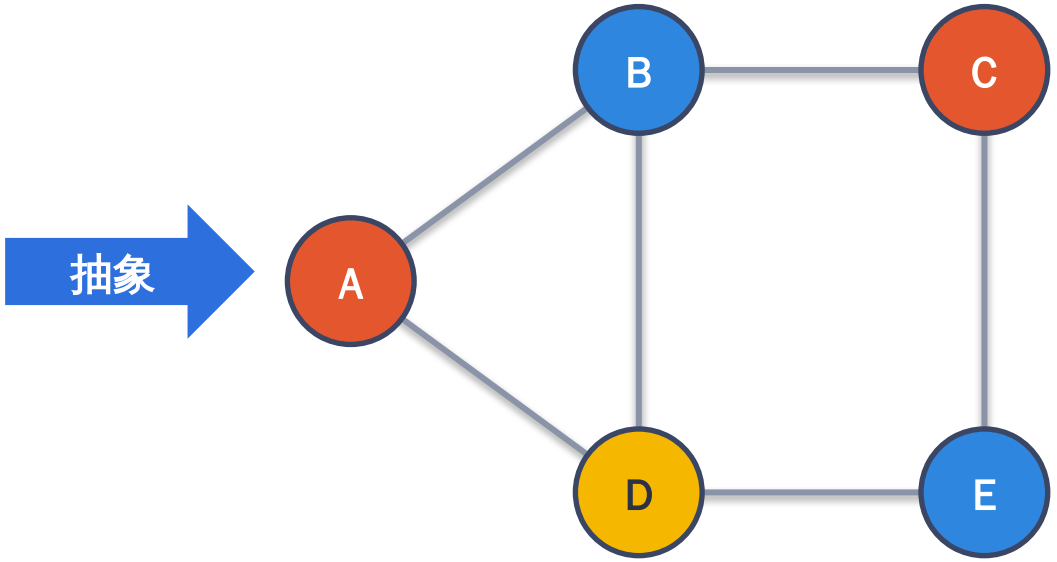
五元环 C5 ：二染色失败，三染色成功

从地图到图：抽象掉形状，只留下关系

给地图上色，相邻区域颜色必须不同——把区域缩成点、相邻连成边，地图就变成了图



地图：形状、面积都不重要



图：顶点 = 区域，边 = 相邻

编译器寄存器分配
变量 → 顶点
同时活跃 → 边
寄存器 → 颜色

教务排考试
课程 → 顶点
有共同学生 → 边
考试时段 → 颜色

无线电频点分配
基站 → 顶点
相邻干扰 → 边
频率 → 颜色

2 色、3 色、4 色：恰好踩在难易分界线上

只多一种颜色，问题从“瞬间可解”掉进“无人会解”

2 色

二分图判定

- 线性时间 O(n+m)
- BFS/DFS 交替染色
- 没有选择，不需要回溯

3 色

NP 完全

- Karp 1972 ： 21 个 NP 完全问题之一
- 限定平面图仍是 NP 完全（Garey–Johnson–Stockmeyer, 1976）
- 验证容易、求解没有已知的快速通用算法

平面图 4 色

四色定理：永远有解

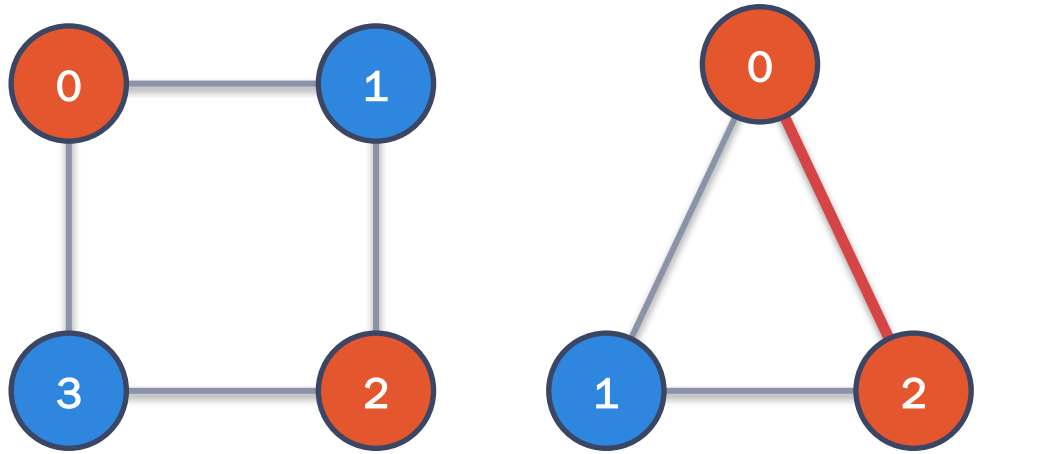
- 1976 年 Appel–Haken 计算机辅助证明
- 存在 $O(n^2)$ 算法
- 所以本项目选三色：三色才是搜索问题的战场

NP 完全 ≠ 每个实例都难，也不表示已证明 P ≠ NP —— 这一点后面用数据说明

任务一：二染色——线性时间，没有选择

BFS 交替染色；奇环必然矛盾

```
two_color(n, adj):
  color ← [-1] × n
  for s in 0..n-1 且 color[s] == -1:   # 每个连通分量
      color[s] ← 0;  queue ← [s]
      while queue 非空:
          u ← queue.pop()
          for v in adj[u]:
              if color[v] == -1:
                  color[v] ← 1 - color[u]; queue.push(v)
              elif color[v] == color[u]:
                  return None            # 奇环
  return color
```



C4 （偶环）：[0,1,0,1] ✓　　K3 （奇环）：必有同色边 ×

- 为什么是线性：每个顶点最多入队一次，每条边从两端各看一次 → O(n+m)
- 为什么不需要回溯：起点定色后，分量内其余颜色被唯一决定
- 孤立点、不连通图：每个未染色顶点重新作起点

网格图为什么一定是二分图？
颜色 =（行号 + 列号）mod 2 。每条边连接相邻两格，行 + 列之和相差 1 ，奇偶必不同。
奇环：绕一圈颜色逐边交替，奇数条边后回到起点却颜色相反 → 矛盾。

任务二：朴素回溯三染色——规则完全钉死

固定顺序、固定颜色顺序、只和已染色邻居比较

- 顶点按 0, 1, …, n−1 固定顺序染色
- 每个顶点依次尝试颜色 0, 1, 2
- 只与已染色邻居判冲突，不向前看
- 冲突就换色；三色都不行 → 回溯到上一个顶点
- 第一个完整方案立即返回
- NODES = 进入 dfs 的次数（dfs(0) 与 v==n 的成功调用都计）

```
three_color_backtrack(n, adj, stats):
  若存在自环 v ∈ adj[v]: return None
  coloring ← [UNCOLORED] × n

  dfs(v):                          # 给顶点 v 染色
      stats.nodes += 1
      if v == n: return 成功
      for c in 0, 1, 2:
          if ∃ 已染色邻居 u: coloring[u] == c:
              continue             # 冲突，换色
          coloring[v] ← c
          if dfs(v + 1): return 成功
          coloring[v] ← UNCOLORED   # 撤销
      return 失败

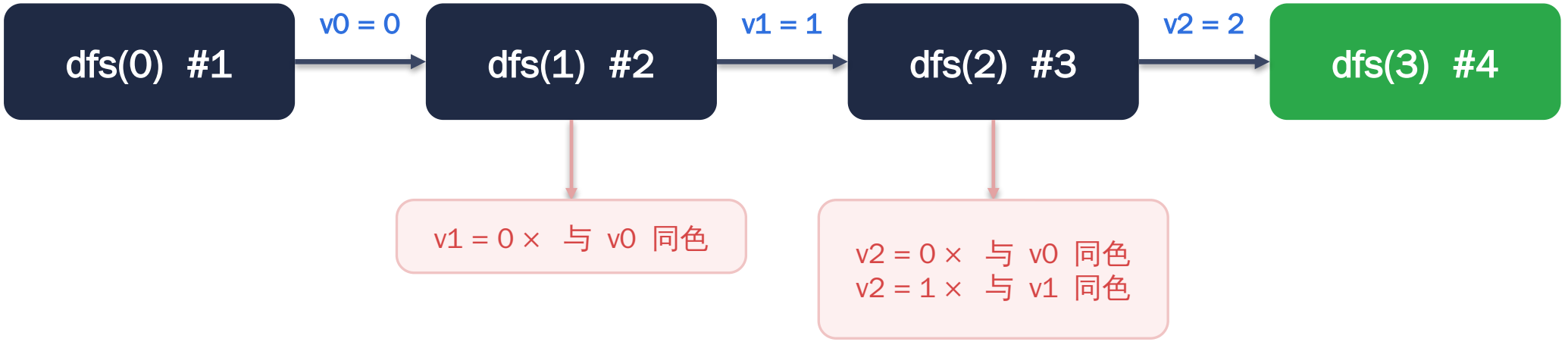
  return coloring if dfs(0) else None
```

撤销时恢复什么？ coloring[v] ← UNCOLORED 。忘记撤销是经典 bug ：残留颜色会污染之后所有分支； nodes 是统计量，不回滚。

回溯树（一）：三角形——有解只需一条路径

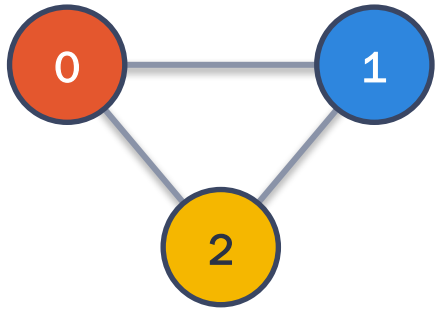
dfs(v) = 给顶点 v 染色；× = 与已染色邻居冲突，直接跳过

v0 = 1 、 v0 = 2 从未尝试：第一个完整方案出现即停止



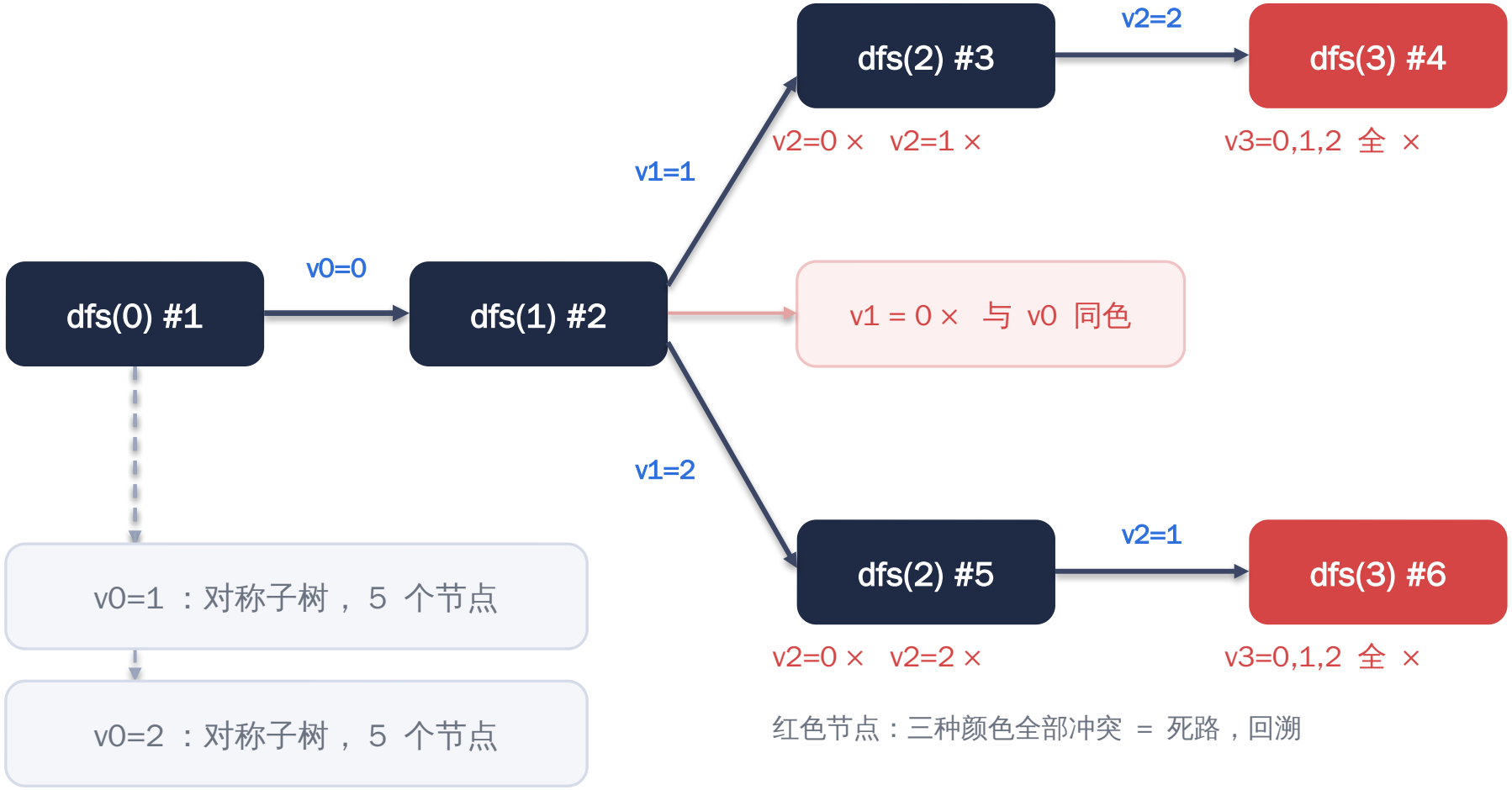
结果：[0, 1, 2] ， NODES = 4 ，没有回溯

“ 有解”的证据 = 一个合法染色，逐边检查即可验证： O(n+m)



回溯树（二）：K4—— 无解必须覆盖全部分支

只画 v0 = 0 的子树；一共 1 + 3 × 5 = 16 个节点



红色节点：三种颜色全部冲突 = 死路，回溯

无解的证据

必须覆盖或正确排除所有分支

共 16 个节点
= 1 + 3 × 5

算法没有利用对称性：
v0=1 、 v0=2 的子树要重新搜完

提前 return 无解是错误的

现场演示： solve 与 bench

同一个求解器，六个经典实例； node 数是对“难度”最直观的度量

```
python src/main.py solve --data data/triangle.txt

python src/main.py solve --data data/k4.txt

python src/main.py solve --data data/grid_4x4.txt --two

python src/main.py bench --naive-only --degree 4.7 \
       --reps 20 --sizes 10 12 14 16 18 20 22 24 26 28
```

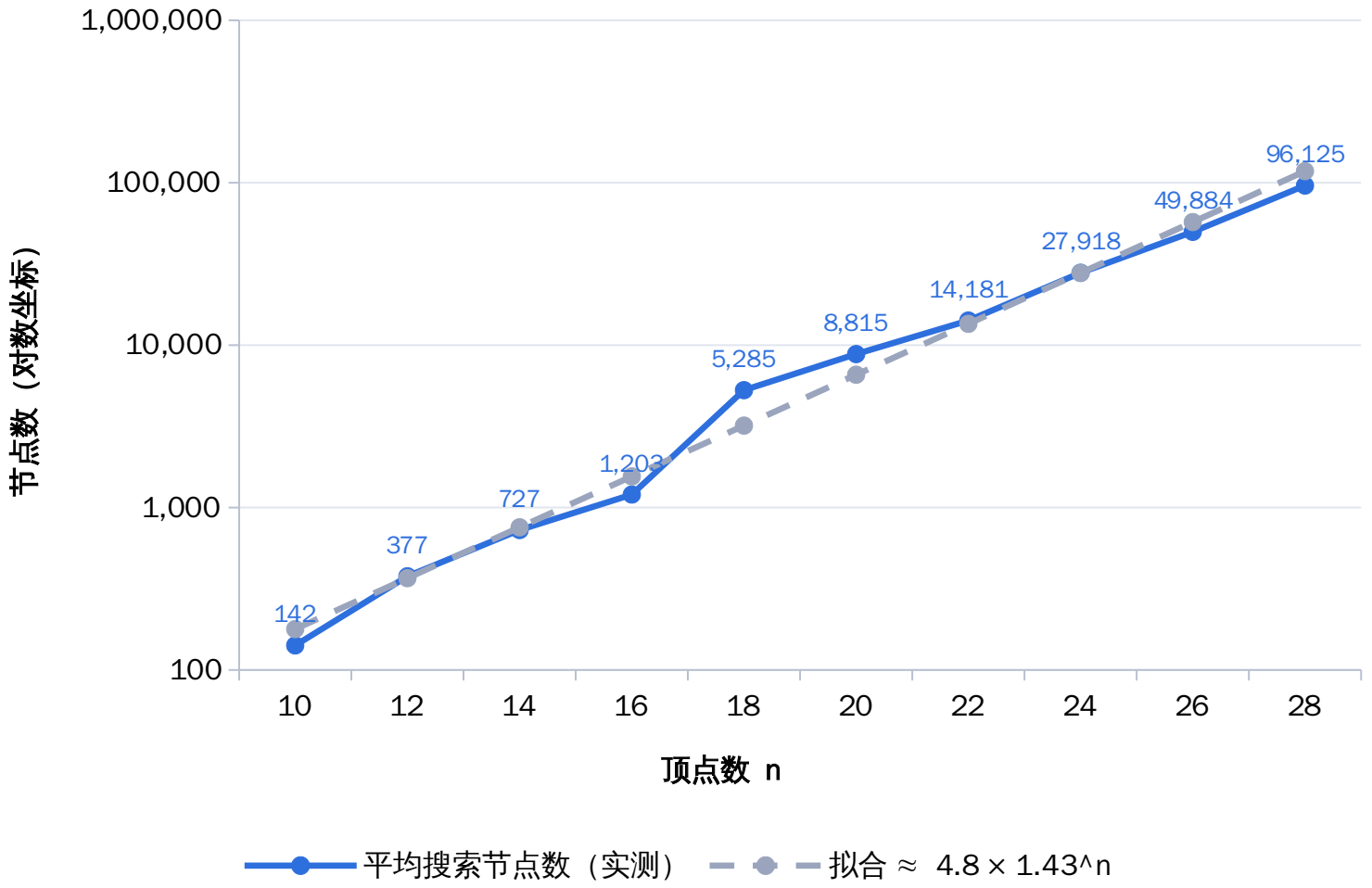
- triangle ：可三染色， [0,1,2] ， NODES = 4
- k4 ：不可三染色， NODES = 16
- grid_4x4 ：二染色成功（二分图）
- bench ：随机图，节点数随 n 增长（下页）

实例	n	m	二染色	三染色	NODES
triangle	3	3	否	可	4
k4	4	6	否	不可	16
cycle5	5	5	否	可	6
grid_4x4	16	24	是	可	17
petersen	10	15	否	可	11
mycielski4	11	20	否	不可	472

mycielski4 只有 11 个顶点，却要 472 个节点才能确认无解 —— 比 K4 （16）多 30 倍

指数爆炸实验：节点数随 n 近似 c^n 增长

随机图，平均度数 4.7 （m = round(4.7n/2)），每个规模 20 个实例取平均，种子 2026–2045



读图要点

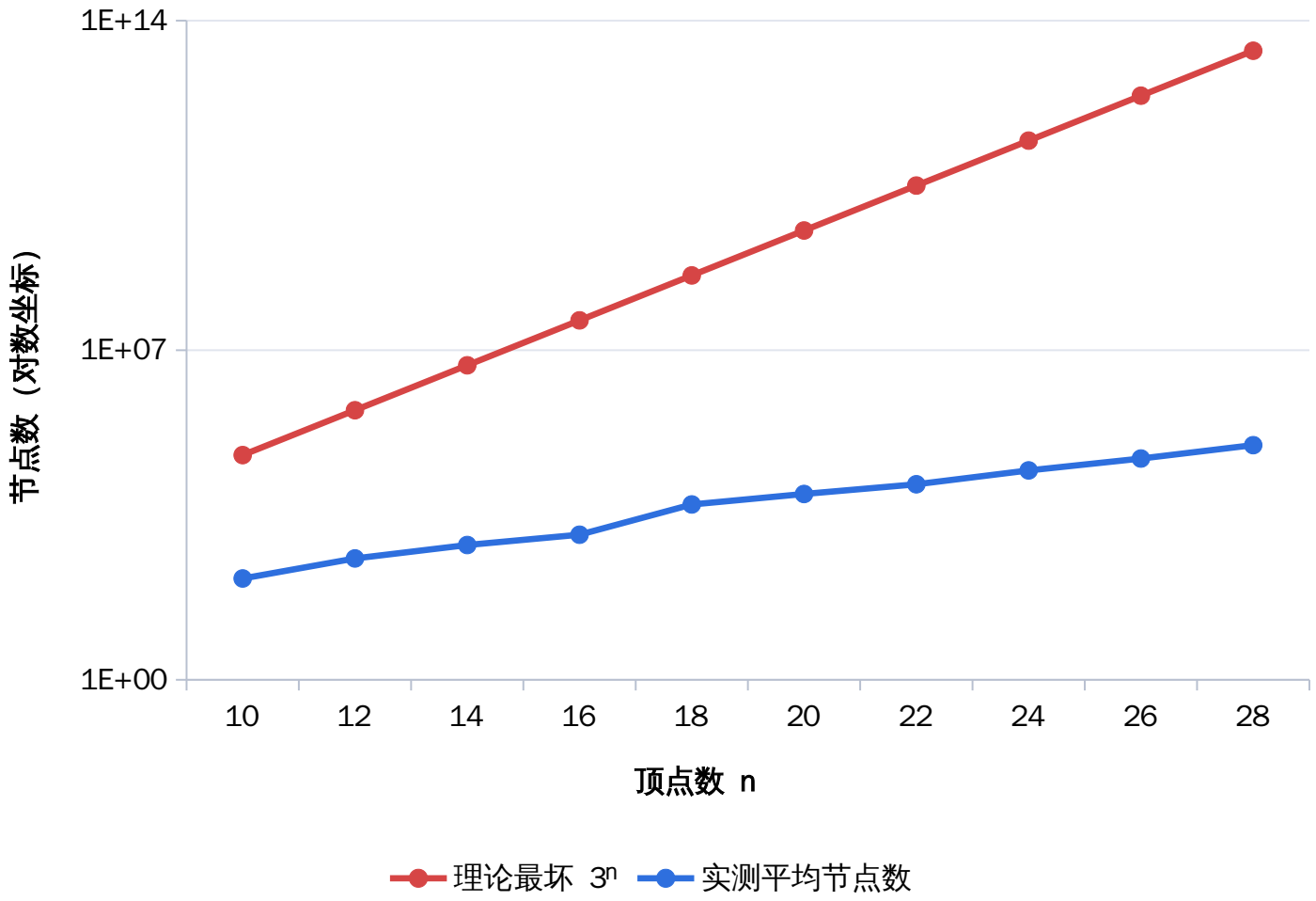
最小二乘拟合： c ≈ 1.43
n = 20 → 28 ：逐步增长率约 1.35
每增加 2 个顶点，节点数约翻一番

n = 28 时平均约 9.6 万个节点、约 0.1 秒（本机 Python）；n 再增大会迅速变慢

仅为 n ≤ 28 的实验趋势，不是复杂度证明

谁在“免费”剪枝？实测只有最坏 3^n 的 4×10^{-9}

冲突检查：一旦与已染色邻居冲突，整棵子树立刻放弃



n = 28

$3^{28} \approx 2.3 \times 10^{13}$

实测 ≈ 9.6×10^{4}

仅为最坏情形的 ≈ 4×10^{-9}

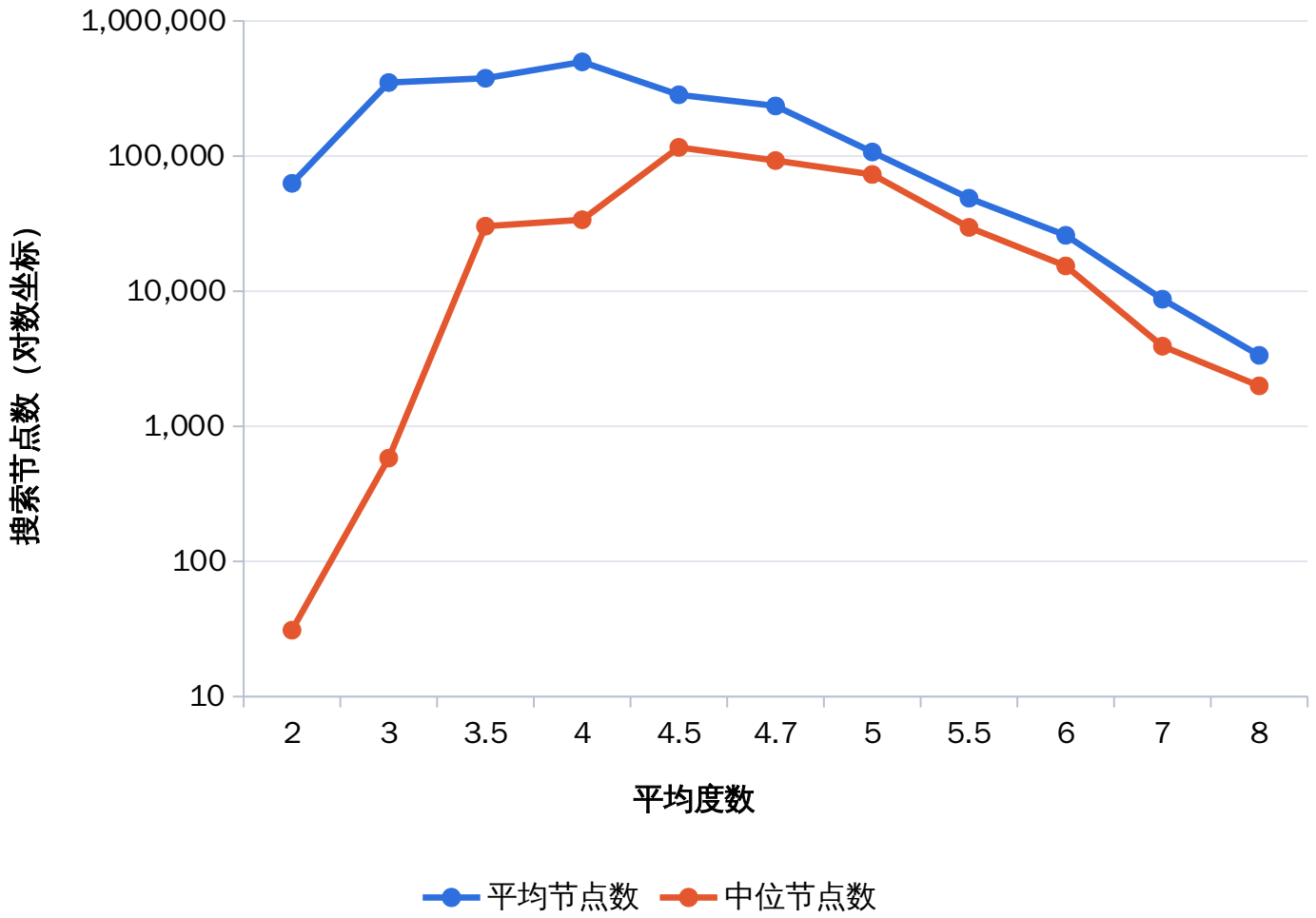
剪枝来自哪里？

每次尝试颜色都先检查已染色邻居；冲突 → 以该顶点为根的整棵子树不再展开

但：底数从 3 降到约 1.43 ，增长仍是指数的

难度先升后降： easy–hard–easy

n = 30 ，每个平均度数 30 个随机实例，纵轴为朴素回溯的搜索节点数

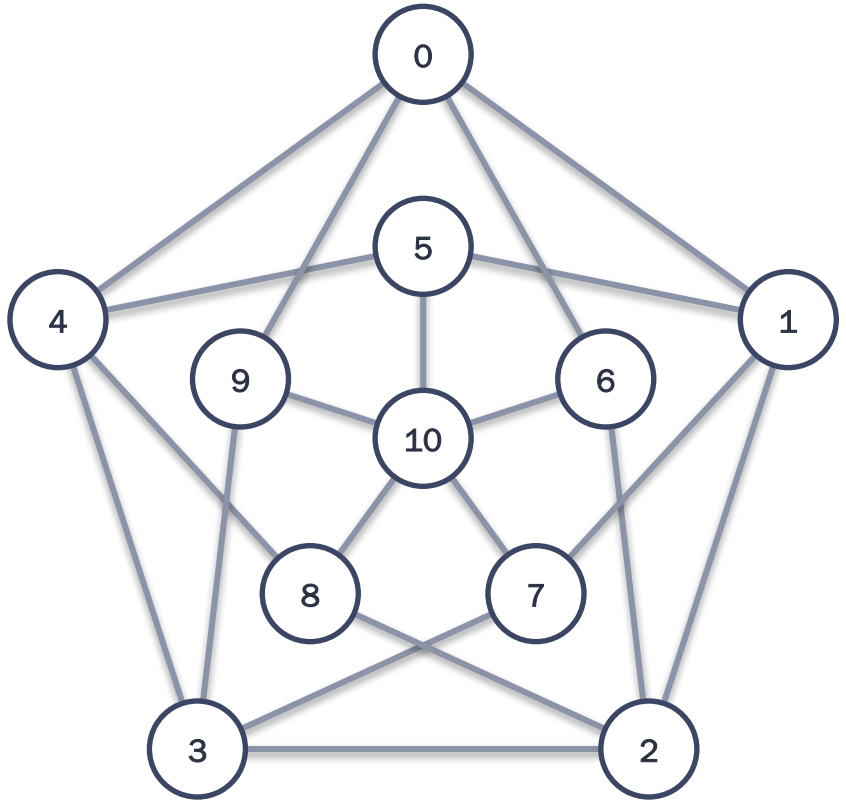


度数	m	均值	中位数	可三染色
2	30	62,933	31	30/30
3	45	350,676	582	29/30
3.5	52	377,192	30,306	26/30
4	60	498,552	33,835	17/30
4.5	68	284,712	116,157	6/30
4.7	70	235,256	92,772	4/30
5	75	107,322	73,174	1/30
5.5	82	48,850	29,707	0/30
6	90	25,948	15,396	0/30
7	105	8,737	3,922	0/30
8	120	3,357	1,995	0/30

最难区约在度数 4–5 （可染 / 不可染分界附近）；稀疏端均值被个别重尾实例拉高

Grötzsch 图：没有三角形，却不能三染色

“难”不一定有显而易见的局部理由



Grötzsch 图： 11 个顶点、 20 条边、无三角形

11 个顶点， 20 条边，不含任何三角形，色数却是 4

朴素回溯要 472 个节点才确认无解（ K4 只需 16 个）

无解没有“一眼可见”的局部理由（不像 K4 的完全子图）

对 NP 完全的启示

有解：给出方案， O(n+m) 即可验证

无解：没有已知的通用办法给出比穷尽搜索更短的证据

实现与验证：怎么知道它是对的

OJ 单文件框架 + 独立实现交叉验证 + C++ 副本逐点对照

OJ 单文件框架

- 只补全 three_color(n, adj) ， I/O 由框架提供
- 输出 SAT / 颜色 / NODES ，全文精确比较
- 自环：直接 UNSAT ， NODES 0
- n 可达 500 ，递归深度需调高上限

本地验收 20/20

- 19 个测试点 + 题面样例全文比较
- 链、偶环（n=500）、 K(40,40)
- Petersen 、 Grötzsch 、 K5 、不连通图
- 随机图 n=28 ： UNSAT ， 224,443 节点 ≈ 0.2 s
- 独立显式栈实现交叉验证 SAT/UNSAT 、方案、 NODES

C++ 副本 68/68

- 与 Python 版逐点对照染色方案与 NODES
- 含 19 个 OJ 测试点、 data/ 实例与 40 个随机图
- 两版规则相同：顶点 0..n−1 、颜色 0,1,2
- check.py ：任务一、二通过

NP 完全 ≠ 每个实例都难：链 / 偶环 / K(40,40) 分别只要 n+1 、 n+1 、 81 个节点；相变区的随机图可能要搜几十万节点

回溯为什么可能很慢？启发式又能买到什么？

我们对终极问题的理解（Basic 阶段的结论 + 下一步）

回溯为什么可能很慢

- 最坏 3^n：每个顶点 3 个选择，搜索树指数增长（实测 $\approx 1.43^n$）
- 固定顺序看不到后面的约束：冲突发现得晚，一次失败要退很远
- 无解必须覆盖所有分支（K4：16 节点；Grötzsch：472 节点）
- 难度取决于实例结构：链 / 偶环几乎不回溯，相变区才爆炸

启发式能买到什么（下一步 Advance）

- MRV：改变选择顺序，先安排最容易卡死的顶点
- 前向检查：染色后立刻收缩邻居的可用颜色，提前发现死路
- 保留全部合法分支 → 仍然精确、完备，答案不变
- 减少的是搜索规模，并不改变最坏情形 NP 完全的结论

（本次 Basic 未实现 MRV / 前向检查）

小结与问答

Q1–Q3 一句话回顾

问题	一句话回答
Q1 怎样判断染色合法？	指定范围内的颜色，逐条边检查两端不同；检查 m 条边， O(n+m)
Q2 为什么二染色可用 BFS/DFS ？	起点定色后整个分量被唯一决定，沿边交替传播；遇同色相邻即失败，奇环必然矛盾
Q3 回溯搜索什么，何时退回？	搜索部分染色方案；与已染色邻居冲突则跳过，三色都不行就退回并恢复 coloring[v]

三个要点

- 有解易验证，无解要穷尽
- NP 完全 ≠ 每个实例都难；难度看结构
- 冲突检查替我们免费剪掉了绝大部分 3^n

参考资料

- Russell & Norvig ，《人工智能：一种现代方法》第 6 章（约束满足问题）
- Karp, Reducibility Among Combinatorial Problems, 1972
- Garey, Johnson & Stockmeyer, 1976 （平面图三染色 NP 完全）
- Cheeseman, Kanefsky & Taylor, Where the Really Hard Problems Are, IJCAI 1991